Module: Python

1. Python Basics

What is Python?

Python is a programming language that's easy to read and write. Think of it like giving instructions to a computer in a way that's almost like speaking English.

Install & Setup IDE

What is an IDE? An IDE (Integrated Development Environment) is like a special text editor for writing code. It's like Microsoft Word, but designed specifically for programming.

Popular Options:

- **VS Code**: Free, lightweight, great for beginners
- **PyCharm**: More features, good for bigger projects
- **Jupyter Notebook**: Great for learning and experimenting

Simple Setup:

1. Download Python from python.org
2. Download VS Code
3. Install Python extension in VS Code
4. You're ready to code!

Syntax - The Rules of Python

Python syntax is like grammar rules for speaking to computers.

```
# This is a comment - Python ignores this line
print("Hello, World!")  # This prints text to screen
```

Key Rules:

- Each instruction goes on a new line
- Use # for comments (notes to yourself)
- Indentation (spaces) matters in Python

Variables - Storing Information

Think of variables like labeled boxes where you store things.

```
# Creating variables (like putting things in labeled boxes)
name = "John"           # Text (string)
age = 25               # Number (integer)
height = 5.8           # Decimal number (float)
is_student = True      # True/False (boolean)

# Using variables
print("My name is", name)
print("I am", age, "years old")
```

Data Types - Different Kinds of Information

Just like in real life, we have different types of information:

```
# Text (String)
favorite_color = "blue"
message = "Python is fun!"

# Whole Numbers (Integer)
number_of_pets = 3
year = 2024

# Decimal Numbers (Float)
temperature = 98.6
price = 19.99

# True/False (Boolean)
is_raining = False
likes_pizza = True

# Check what type something is
print(type(name))        # <class 'str'>
print(type(age))         # <class 'int'>
```

Operators - Doing Math and Comparisons

Arithmetic Operators (Math):

```
# Basic math
a = 10
b = 3

addition = a + b        # 13
subtraction = a - b     # 7
multiplication = a * b  # 30
division = a / b        # 3.33...
floor_division = a // b # 3 (whole number division)
remainder = a % b       # 1 (what's left over)
power = a ** b          # 1000 (10 to the power of 3)
```

Comparison Operators:

x = 5
y = 10

print(x == y) # False (equal to)
print(x != y) # True (not equal to)
print(x < y) # True (less than)
print(x > y) # False (greater than)
print(x <= y) # True (less than or equal to)
print(x >= y) # False (greater than or equal to)

Conditional Statements - Making Decisions

Like asking "if this, then do that":

```
x = 5
y = 10

print(x == y)    # False (equal to)
print(x != y)    # True (not equal to)
print(x < y)     # True (less than)
print(x > y)     # False (greater than)
print(x <= y)    # True (less than or equal to)
print(x >= y)    # False (greater than or equal to)g to vote.")
```

```
# Multiple conditions
score = 85

if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
else:
    print("Grade: F")
```

Real-world example:

```
weather = "sunny"
temperature = 35

if weather == "sunny" and temperature > 30:
    print("Perfect day for a picnic!")
elif weather == "rainy":
    print("Take an umbrella!")
else:
    print("Check the weather again.")
```

Loops - Repeating Actions

For Loop (when you know how many times):

```
# Print numbers 1 to 5
for i in range(1, 6):
    print("Number:", i)

# Loop through a list of items
fruits = ["apple", "banana", "orange"]
for fruit in fruits:
    print("I like", fruit)
```

While Loop (keep going until something changes):

```
# Count down from 5
countdown = 5
while countdown > 0:
    print(countdown)
    countdown = countdown - 1
print("Blast off!")

# Simple guessing game
secret_number = 7
guess = 0

while guess != secret_number:
    guess = int(input("Guess a number (1-10): "))
    if guess == secret_number:
        print("Correct!")
    else:
        print("Try again!")
```

2. Data Structures - Organizing Information

Lists - Ordered Collections

Think of lists like shopping lists - items in order, can be changed:

```
# Creating lists
shopping_list = ["milk", "bread", "eggs", "cheese"]
numbers = [1, 2, 3, 4, 5]
mixed_list = ["John", 25, True, 98.6]

# Accessing items (counting starts from 0)
print(shopping_list[0])    # "milk" (first item)
print(shopping_list[1])    # "bread" (second item)
print(shopping_list[-1])   # "cheese" (last item)

# Changing items
shopping_list[0] = "almond milk"
print(shopping_list)

# Adding items
```

```
shopping_list.append("apples")      # Add to end
shopping_list.insert(0, "bananas") # Add at beginning

# Removing items
shopping_list.remove("bread")       # Remove specific item
last_item = shopping_list.pop()     # Remove and get last item

# List operations
print(len(shopping_list))           # How many items?
print("milk" in shopping_list)      # Is "milk" in the list?
```

Tuples - Fixed Collections

Like lists, but you can't change them once created:

```
# Creating tuples
coordinates = (10, 20)
rgb_color = (255, 128, 0)
person_info = ("John", 25, "Engineer")

# Accessing items (same as lists)
print(coordinates[0])    # 10
print(coordinates[1])    # 20

# Tuples are useful for data that shouldn't change
months = ("Jan", "Feb", "Mar", "Apr", "May", "Jun",
          "Jul", "Aug", "Sep", "Oct", "Nov", "Dec")

# You can't do this with tuples:
# months[0] = "January"  # This would cause an error!
```

Sets - Unique Collections

Sets automatically remove duplicates and have no order:

```
# Creating sets
unique_numbers = {1, 2, 3, 4, 5}
colors = {"red", "green", "blue", "red"}  # Duplicate "red" is removed
print(colors)  # {'red', 'green', 'blue'}
```

```python
# Converting list to set (removes duplicates)
numbers_with_duplicates = [1, 2, 2, 3, 3, 3, 4]
unique_numbers = set(numbers_with_duplicates)
print(unique_numbers)  # {1, 2, 3, 4}

# Set operations
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}

print(set1.union(set2))        # {1, 2, 3, 4, 5, 6} (combine)
print(set1.intersection(set2)) # {3, 4} (common items)
```

Dictionaries - Key-Value Pairs

Like a phone book - you look up a name (key) to find a number (value):

```python
# Creating dictionaries
student = {
    "name": "Alice",
    "age": 20,
    "grade": "A",
    "courses": ["Math", "Science", "English"]
}

# Accessing values
print(student["name"])     # "Alice"
print(student["age"])      # 20

# Adding/changing values
student["email"] = "alice@email.com"  # Add new key-value
student["age"] = 21                   # Change existing value

# Useful dictionary methods
print(student.keys())      # Get all keys
print(student.values())    # Get all values
print(student.items())     # Get key-value pairs
```

```python
# Real-world example: Phone book
phone_book = {
    "Mom": "555-1234",
    "Dad": "555-5678",
    "Best Friend": "555-9999"
}

# Look up a number
contact = "Mom"
if contact in phone_book:
    print(f"{contact}'s number is {phone_book[contact]}")
```

3. Functions & Modularization

What are Functions?

Functions are like recipes - you give them ingredients (inputs), and they give you a dish (output):

```python
# Simple function
def greet():
    print("Hello, World!")

# Call the function
greet()  # Output: Hello, World!

# Function with input (parameters)
def greet_person(name):
    print(f"Hello, {name}!")

greet_person("Alice")  # Output: Hello, Alice!
greet_person("Bob")    # Output: Hello, Bob!

# Function that returns a value
def add_numbers(a, b):
    result = a + b
    return result

sum_result = add_numbers(5, 3)
print(sum_result)  # 8
```

More Function Examples

```
# Function to calculate area of rectangle
def rectangle_area(length, width):
    area = length * width
    return area

# Function to check if number is even
def is_even(number):
    if number % 2 == 0:
        return True
    else:
        return False

# Using the functions
room_area = rectangle_area(12, 10)
print(f"Room area: {room_area} square feet")

number = 7
if is_even(number):
    print(f"{number} is even")
else:
    print(f"{number} is odd")
```

Pattern Programs Examples

```
# Function to print a triangle pattern
def print_triangle(rows):
    for i in range(1, rows + 1):
        # Print spaces
        for space in range(rows - i):
            print(" ", end="")
        # Print stars
        for star in range(i):
            print("* ", end="")
        print()  # New line

print_triangle(5)
# Output:
#     *
#    * *
#   * * *
```

```
#  * * * *
# * * * * *
```

```
# Function to print number pattern
def print_number_pattern(rows):
    for i in range(1, rows + 1):
        for j in range(1, i + 1):
            print(j, end=" ")
        print()

print_number_pattern(4)
# Output:
# 1
# 1 2
# 1 2 3
# 1 2 3 4
```

4. File Handling & OS Interaction

Reading and Writing Files

Think of file handling like opening a book, reading it, writing in it, and closing it:

```
# Writing to a file
def write_to_file():
    # Open file for writing (creates new file or overwrites existing)
    with open("my_notes.txt", "w") as file:
        file.write("This is my first line\n")
        file.write("This is my second line\n")
        file.write("Python is awesome!")
    print("File written successfully!")

write_to_file()

# Reading from a file
def read_from_file():
    try:
        with open("my_notes.txt", "r") as file:
            content = file.read()
            print("File content:")
```

```
            print(content)
    except FileNotFoundError:
        print("File not found!")

read_from_file()

# Reading line by line
def read_file_lines():
    with open("my_notes.txt", "r") as file:
        for line_number, line in enumerate(file, 1):
            print(f"Line {line_number}: {line.strip()}")

read_file_lines()
```

Working with Different File Types

```
# Appending to a file (adding to the end)
def add_to_file(filename, new_content):
    with open(filename, "a") as file:  # "a" means append
        file.write(f"\n{new_content}")

add_to_file("my_notes.txt", "This line was added later")

# Working with CSV-like data
def write_student_data():
    students = [
        "John,20,Math",
        "Alice,19,Science",
        "Bob,21,English"
    ]

    with open("students.txt", "w") as file:
        file.write("Name,Age,Subject\n")  # Header
        for student in students:
            file.write(student + "\n")

write_student_data()
```

Basic OS Module Usage

The `os` module helps you interact with your computer's operating system:

```
import os

# Get current directory (where your Python script is)
current_dir = os.getcwd()
print("Current directory:", current_dir)

# List files in current directory
files = os.listdir()
print("Files in current directory:")
for file in files:
    print(f"  - {file}")

# Check if file exists
filename = "my_notes.txt"
if os.path.exists(filename):
    print(f"{filename} exists!")
else:
    print(f"{filename} does not exist!")

# Get file size
if os.path.exists(filename):
    size = os.path.getsize(filename)
    print(f"File size: {size} bytes")
```

Basic sys Module Usage

The `sys` module gives you access to system-specific parameters:

```
import sys

# Get Python version
print("Python version:", sys.version)

# Get command line arguments (when running script from terminal)
print("Script name:", sys.argv[0])
```

```python
if len(sys.argv) > 1:
    print("Arguments:", sys.argv[1:])

# Exit the program with a message
def exit_with_message(message):
    print(message)
    sys.exit(1)  # 1 means error, 0 means success

# Example usage (uncomment to test):
# exit_with_message("Something went wrong!")
```

5. Error Handling & Logging

Understanding Errors

Errors are like warning signs - they tell you something went wrong:

```python
# Common error example
def divide_numbers(a, b):
    result = a / b  # What if b is 0?
    return result

# This would crash:
# result = divide_numbers(10, 0)  # ZeroDivisionError!
```

Try/Except Blocks - Handling Errors Gracefully

```python
# Basic error handling
def safe_divide(a, b):
    try:
        result = a / b
        return result
    except ZeroDivisionError:
        print("Error: Cannot divide by zero!")
        return None

# Test the function
print(safe_divide(10, 2))   # 5.0
print(safe_divide(10, 0))   # Error message, returns None
```

```python
# Handling multiple types of errors
def safe_convert_to_int(value):
    try:
        number = int(value)
        return number
    except ValueError:
        print(f"Error: '{value}' is not a valid number!")
        return None
    except Exception as e:
        print(f"Unexpected error: {e}")
        return None

# Test the function
print(safe_convert_to_int("42"))     # 42
print(safe_convert_to_int("hello"))  # Error message
print(safe_convert_to_int("3.14"))   # Error message
```

More Practical Error Handling

```python
# File handling with error checking
def read_file_safely(filename):
    try:
        with open(filename, "r") as file:
            content = file.read()
            return content
    except FileNotFoundError:
        print(f"Error: File '{filename}' not found!")
        return None
    except PermissionError:
        print(f"Error: No permission to read '{filename}'!")
        return None
    except Exception as e:
        print(f"Unexpected error reading file: {e}")
        return None

# User input validation
def get_age_from_user():
    while True:
        try:
            age = int(input("Enter your age: "))
```

```
            if age < 0:
                print("Age cannot be negative!")
                continue
            if age > 150:
                print("That's too old! Please enter a realistic age.")
                continue
            return age
        except ValueError:
            print("Please enter a valid number!")
        except KeyboardInterrupt:
            print("\nProgram cancelled by user.")
            return None

# age = get_age_from_user()
# print(f"Your age is: {age}")
```

Basic Logging

Logging is like keeping a diary of what your program does:

```
import logging

# Set up basic logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)

def calculate_with_logging(a, b, operation):
    logging.info(f"Starting calculation: {a} {operation} {b}")

    try:
        if operation == "+":
            result = a + b
        elif operation == "-":
            result = a - b
        elif operation == "*":
            result = a * b
        elif operation == "/":
            if b == 0:
                raise ZeroDivisionError("Division by zero!")
```

```
            result = a / b
        else:
            raise ValueError(f"Unknown operation: {operation}")

        logging.info(f"Calculation successful: {result}")
        return result

    except Exception as e:
        logging.error(f"Calculation failed: {e}")
        return None

# Test the function
print(calculate_with_logging(10, 5, "+"))   # 15
print(calculate_with_logging(10, 0, "/"))   # None (with error log)
```

6. Virtual Environments & Package Management

What are Virtual Environments?

Imagine you have different toolboxes for different projects. Virtual environments are like separate toolboxes for your Python projects:

```
# Create a virtual environment (run in terminal/command prompt)
python -m venv my_project_env

# Activate it (Windows)
my_project_env\Scripts\activate

# Activate it (Mac/Linux)
source my_project_env/bin/activate

# When activated, you'll see (my_project_env) in your terminal
```

Package Management with pip

pip is like an app store for Python packages:

```
# Install a package
pip install requests
```

```
# Install a specific version
pip install requests==2.28.0

# Install multiple packages
pip install requests pandas numpy

# See what's installed
pip list

# Save your project's packages to a file
pip freeze > requirements.txt

# Install packages from requirements.txt
pip install -r requirements.txt

# Uninstall a package
pip uninstall requests
```

Creating and Using requirements.txt

```
# Example requirements.txt file content:
requirements_content = """
requests==2.28.1
pandas==1.5.0
numpy==1.23.0
matplotlib==3.6.0
"""

# Create requirements.txt file
def create_requirements_file():
    with open("requirements.txt", "w") as file:
        file.write(requirements_content.strip())
    print("requirements.txt created!")

create_requirements_file()

# Function to read requirements
def read_requirements():
    try:
        with open("requirements.txt", "r") as file:
            packages = file.readlines()
```

```
        print("Required packages:")
        for package in packages:
            if package.strip():  # Skip empty lines
                print(f"  - {package.strip()}")

    except FileNotFoundError:
        print("requirements.txt not found!")

read_requirements()
```

The examples are designed to be:

- **Practical**: Real-world scenarios you might encounter
- **Progressive**: Starting simple and gradually increasing complexity
- **Hands-on**: Code you can actually run and modify
- **Well-commented**: Explanations for every important concept

Start with the basics and work your way through each section, practicing the examples and experimenting with modifications to deepen your understanding!

Python Practice Problems

Problem 1: Personal Information Display

Problem Statement: Create a program that stores your personal information (name, age, height, and whether you're a student) in variables and displays them nicely.

```
# Store personal information
name = "John Doe"
age = 20
height = 5.8
is_student = True

# Display information
print("Personal Information:")
print("Name:", name)
print("Age:", age, "years old")
print("Height:", height, "feet")
print("Student status:", is_student)

# Calculate birth year (simple math)
current_year = 2024
birth_year = current_year - age
print("You were born in:", birth_year)
```

Problem 2: Simple Calculator

Problem Statement: Create a simple calculator that performs basic math operations on two numbers.

```
# Get two numbers
num1 = 15
num2 = 4

# Perform calculations
addition = num1 + num2
subtraction = num1 - num2
```

```
multiplication = num1 * num2
division = num1 / num2

# Display results
print("Calculator Results:")
print(f"{num1} + {num2} = {addition}")
print(f"{num1} - {num2} = {subtraction}")
print(f"{num1} × {num2} = {multiplication}")
print(f"{num1} ÷ {num2} = {division}")
```

Problem 3: Age Category Checker

Problem Statement: Write a program that determines if a person is a child, teenager, adult, or senior based on their age.

```
# Check age category
age = 17

if age < 13:
    category = "Child"
elif age < 20:
    category = "Teenager"
elif age < 60:
    category = "Adult"
else:
    category = "Senior"

print(f"Age {age} is classified as: {category}")

# Bonus: Check if they can vote
if age >= 18:
    print("You can vote!")
else:
    years_to_wait = 18 - age
    print(f"You can vote in {years_to_wait} years.")
```

Problem 4: Simple Grade Calculator

Problem Statement: Calculate a letter grade based on a numeric score.

```
# Student score
score = 85

# Determine grade
if score >= 90:
    grade = "A"
    message = "Excellent work!"
elif score >= 80:
    grade = "B"
    message = "Good job!"
elif score >= 70:
    grade = "C"
    message = "Fair work."
elif score >= 60:
    grade = "D"
    message = "Needs improvement."
else:
    grade = "F"
    message = "Please study harder."

print(f"Score: {score}")
print(f"Grade: {grade}")
print(f"Comment: {message}")
```

Problem 5: Number Counter

Problem Statement: Create a program that counts numbers and prints them in different patterns.

```
# Count from 1 to 10
print("Counting up:")
for i in range(1, 11):
    print(i, end=" ")
print()  # New line

# Count from 10 to 1
```

```
print("Counting down:")
for i in range(10, 0, -1):
    print(i, end=" ")
print()  # New line

# Count even numbers from 2 to 20
print("Even numbers:")
for i in range(2, 21, 2):
    print(i, end=" ")
print()
```

Problem 6: Simple Patterns

Problem Statement: Create simple star patterns using loops.

```
# Pattern 1: Right triangle
print("Right Triangle Pattern:")
for row in range(1, 6):
    for star in range(row):
        print("*", end="")
    print()  # New line after each row

print()  # Empty line

# Pattern 2: Square pattern
print("Square Pattern:")
for row in range(4):
    for col in range(4):
        print("*", end=" ")
    print()  # New line after each row
```

Problem 7: Favorite Things List

Problem Statement: Create and manipulate a list of your favorite things.

```
# Create a list of favorite foods
favorite_foods = ["pizza", "ice cream", "burgers", "pasta"]

# Display the list
print("My favorite foods:")
for i, food in enumerate(favorite_foods):
    print(f"{i + 1}. {food}")

# Add a new favorite
favorite_foods.append("chocolate")
print("\nAfter adding chocolate:")
print(favorite_foods)

# Remove an item
favorite_foods.remove("burgers")
print("\nAfter removing burgers:")
print(favorite_foods)

# Check if item exists
if "pizza" in favorite_foods:
    print("\nPizza is still in my favorites!")

print(f"\nTotal favorite foods: {len(favorite_foods)}")
```

Problem 8: Number List Operations

Problem Statement: Work with a list of numbers to find basic statistics.

```
# List of test scores
scores = [85, 92, 78, 96, 88, 76, 89, 94]

# Display all scores
print("Test Scores:", scores)

# Find basic statistics
highest_score = max(scores)
lowest_score = min(scores)
```

```
total_scores = sum(scores)
average_score = total_scores / len(scores)

print(f"Highest Score: {highest_score}")
print(f"Lowest Score: {lowest_score}")
print(f"Average Score: {average_score:.1f}")

# Count scores above average
above_average = 0
for score in scores:
    if score > average_score:
        above_average += 1

print(f"Scores above average: {above_average}")
```

Problem 9: Simple Functions Practice

Problem Statement: Create basic functions to perform common tasks.

```
# Function to greet someone
def greet_person(name):
    return f"Hello, {name}! How are you today?"

# Function to calculate area of rectangle
def rectangle_area(length, width):
    area = length * width
    return area

# Function to check if number is even or odd
def check_even_odd(number):
    if number % 2 == 0:
        return "even"
    else:
        return "odd"

# Test the functions
print(greet_person("Alice"))
print(greet_person("Bob"))
```

```
room_area = rectangle_area(10, 12)
print(f"Room area: {room_area} square feet")

num = 7
result = check_even_odd(num)
print(f"The number {num} is {result}")
```

Problem 10: Temperature Converter Function

Problem Statement: Create functions to convert between Celsius and Fahrenheit.

```
def celsius_to_fahrenheit(celsius):
    """Convert Celsius to Fahrenheit"""
    fahrenheit = (celsius * 9/5) + 32
    return fahrenheit

def fahrenheit_to_celsius(fahrenheit):
    """Convert Fahrenheit to Celsius"""
    celsius = (fahrenheit - 32) * 5/9
    return celsius

# Test conversions
temp_c = 25
temp_f = celsius_to_fahrenheit(temp_c)
print(f"{temp_c}°C = {temp_f}°F")

temp_f2 = 77
temp_c2 = fahrenheit_to_celsius(temp_f2)
print(f"{temp_f2}°F = {temp_c2:.1f}°C")

# Common temperatures
print("\nCommon Temperature Conversions:")
common_temps = [0, 25, 30, 37, 100]
for temp in common_temps:
    converted = celsius_to_fahrenheit(temp)
    print(f"{temp}°C = {converted}°F")
```

Problem 11: Student Information Dictionary

Problem Statement: Store and display student information using a dictionary.

```
# Create student record
student = {
    "name": "Emma Wilson",
    "age": 16,
    "grade": "10th",
    "subjects": ["Math", "English", "Science", "History"],
    "gpa": 3.8
}

# Display student information
print("Student Information:")
print(f"Name: {student['name']}")
print(f"Age: {student['age']}")
print(f"Grade: {student['grade']}")
print(f"GPA: {student['gpa']}")

print("Subjects:")
for subject in student["subjects"]:
    print(f"  - {subject}")

# Update information
student["age"] = 17
student["gpa"] = 3.9
print(f"\nUpdated Age: {student['age']}")
print(f"Updated GPA: {student['gpa']}")
```

Problem 12: Simple Inventory System

Problem Statement: Create a basic inventory system using a dictionary.

```
# Store inventory
inventory = {
    "apples": 50,
    "bananas": 30,
    "oranges": 25,
    "grapes": 15
```

```
}

# Display inventory
print("Current Inventory:")
for item, quantity in inventory.items():
    print(f"{item.title()}: {quantity}")

# Check specific item
item_to_check = "apples"
if item_to_check in inventory:
    print(f"\nWe have {inventory[item_to_check]} {item_to_check}")
else:
    print(f"\nWe don't have {item_to_check} in stock")

# Update inventory (sell some items)
inventory["apples"] -= 10
inventory["bananas"] -= 5

print("\nAfter sales:")
for item, quantity in inventory.items():
    print(f"{item.title()}: {quantity}")
```

Problem 13: Safe Number Input

Problem Statement: Create a program that safely gets number input from user with error handling.

```
def get_number_safely():
    """Get a number from user with error handling"""
    while True:
        try:
            user_input = input("Enter a number: ")
            number = float(user_input)
            return number
        except ValueError:
            print("That's not a valid number! Please try again.")

def safe_division():
    """Perform division with error handling"""
```

```
    print("Simple Division Calculator")

    try:
        num1 = get_number_safely()
        num2 = get_number_safely()

        if num2 == 0:
            print("Error: Cannot divide by zero!")
        else:
            result = num1 / num2
            print(f"{num1} ÷ {num2} = {result}")

    except Exception as e:
        print(f"An error occurred: {e}")

# Run the safe division
safe_division()
```

Problem 14: List Operations with Error Handling

Problem Statement: Work with lists while handling potential errors.

```
def safe_list_operations():
    """Demonstrate safe list operations"""
    numbers = [10, 20, 30, 40, 50]

    print("Numbers list:", numbers)

    # Safe list access
    try:
        index = int(input("Enter index to access (0-4): "))
        value = numbers[index]
        print(f"Value at index {index}: {value}")
    except ValueError:
        print("Please enter a valid number!")
    except IndexError:
        print(f"Index out of range! Valid indexes are 0 to
{len(numbers)-1}")

    # Safe list modification
```

```
    try:
        new_value = int(input("Enter a number to add: "))
        numbers.append(new_value)
        print("Updated list:", numbers)
    except ValueError:
        print("Please enter a valid number!")

# Run safe list operations
safe_list_operations()
```

Problem 15: Basic File Writing and Reading

Problem Statement: Create a program that writes to a file and reads from it with basic error handling.

```
def write_to_file():
    """Write some text to a file"""
    try:
        filename = "my_notes.txt"

        # Write to file
        with open(filename, "w") as file:
            file.write("This is my first note.\n")
            file.write("Python is fun to learn!\n")
            file.write("File handling is useful.\n")

        print(f"Successfully wrote to {filename}")

    except Exception as e:
        print(f"Error writing to file: {e}")

def read_from_file():
    """Read text from a file"""
    try:
        filename = "my_notes.txt"

        with open(filename, "r") as file:
            content = file.read()
            print("File contents:")
```

```
            print(content)

    except FileNotFoundError:
        print("File not found! Please create the file first.")
    except Exception as e:
        print(f"Error reading file: {e}")

def count_lines_in_file():
    """Count lines in a file"""
    try:
        filename = "my_notes.txt"

        with open(filename, "r") as file:
            lines = file.readlines()
            line_count = len(lines)

        print(f"The file has {line_count} lines")

        # Show each line with line number
        for i, line in enumerate(lines, 1):
            print(f"Line {i}: {line.strip()}")

    except FileNotFoundError:
        print("File not found!")
    except Exception as e:
        print(f"Error reading file: {e}")

# Run file operations
print("1. Writing to file...")
write_to_file()

print("\n2. Reading from file...")
read_from_file()

print("\n3. Counting lines...")
count_lines_in_file()
```

Remember: Every expert was once a beginner. Take your time, practice regularly, and don't be afraid to make mistakes - they're part of learning!